Scaling

CSI4106 Introduction to Artificial Intelligence

Marcel Turcotte

2026-10-05

This independently executable notebook expands the scaling example from Lecture 10. It focuses on how feature scales alter the geometry used by nearest-neighbour methods.

Scenario

Consider a house-price prediction task using **k-nearest neighbours (KNN) regression** with two features:

- x_1: number of rooms (small scale)
- x_2: square footage (large scale)

The three examples **a**, **b**, and **c** are chosen so that:

- **Without scaling**, **a** is closer to **b** (because square footage dominates).
- **With scaling (z-score)**, **a** becomes closer to **c** (rooms difference matters after rescaling).

Data (three houses)

```
import numpy as np
import pandas as pd

# Three examples (rooms, sqft); prices only for b and c (training)
point_names = ["a", "b", "c"]
X = np.array([
    [4, 1500.0],  # a (query)
    [8, 1520.0],  # b (train)
    [4, 1300.0],  # c (train)
], dtype=float)
```

```
prices = pd.Series([np.nan, 520_000, 390_000], index=point_names, name="price")

df = pd.DataFrame(X, columns=["rooms", "sqft"], index=point_names)
display(df)
display(prices.to_frame())
```

	rooms	sqft
a	4.0	1500.0
b	8.0	1520.0
c	4.0	1300.0

	price
a	NaN
b	520000.0
c	390000.0

Examples **b** and **c** form the training set. Example **a** is the query whose price will be predicted. This deliberately tiny dataset illustrates geometry rather than realistic model estimation.

Euclidean distances (unscaled)

The squared Euclidean distance between u and v is

$$\|u - v\|_2^2 = \sum_j (u_j - v_j)^2.$$

Euclidean distance is its square root, $\|u - v\|_2$; both expressions rank neighbours in the same order.

When one feature has a much larger scale (e.g., square footage), it can dominate the sum.

```
from sklearn.metrics import pairwise_distances

dist_unscaled = pd.DataFrame(
    pairwise_distances(df.values, metric="euclidean"),
    index=df.index, columns=df.index
)
dist_unscaled
```

	a	b	c
a	0.000000	20.396078	200.000000
b	20.396078	0.000000	220.036361
c	200.000000	220.036361	0.000000

```
print("Nearest to 'a' (unscaled):", dist_unscaled.loc["a"].drop("a").idxmin())
```

```
Nearest to 'a' (unscaled): b
```

Result: **a** is nearest to **b** because the square-footage difference dominates the distance.

Proper scaling for modeling (fit scaler on the training set)

For a leakage-safe workflow, compute scaling parameters on the training data (**b**, **c**) only, then transform both the training data and query:

$$z(x) = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}.$$

```
from sklearn.preprocessing import StandardScaler

train_idx = ["b", "c"]
query_idx = ["a"]

scaler = StandardScaler()

scaler.fit(df.loc[train_idx])     # fit only on training points

Z = pd.DataFrame(
    scaler.transform(df),
    columns=df.columns, index=df.index
)

Z
```

	rooms	sqft
a	-1.0	0.818182
b	1.0	1.000000

	rooms	sqft
c	-1.0	-1.000000

Euclidean distances (after scaling)

```
dist_scaled = pd.DataFrame(
    pairwise_distances(Z.values, metric="euclidean"),
    index=Z.index, columns=Z.index
)
dist_scaled
```

	a	b	c
a	0.000000	2.008247	1.818182
b	2.008247	0.000000	2.828427
c	1.818182	2.828427	0.000000

```
print("Nearest to 'a' (scaled):", dist_scaled.loc["a"].drop("a").idxmin())
```

```
Nearest to 'a' (scaled): c
```

Now: **a** is nearest to **c** because the room-count difference matters once the features have comparable scales.

KNN regressor: flip in the prediction

A 1-NN regressor predicts the price of the nearest training example. Compare the same estimator **with and without** scaling.

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.pipeline import Pipeline

X_train = df.loc[train_idx].values          # b, c
y_train = prices.loc[train_idx].values      # prices for b, c
X_query = df.loc[query_idx].values          # a

# 1) No scaling
```

```
knn_plain = KNeighborsRegressor(n_neighbors=1, metric="euclidean")
knn_plain.fit(X_train, y_train)
pred_plain = knn_plain.predict(X_query)[0]

# 2) With scaling (pipeline fits scaler only on training, then KNN on scaled)
knn_scaled = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsRegressor(n_neighbors=1, metric="euclidean"))
])
knn_scaled.fit(X_train, y_train)
pred_scaled = knn_scaled.predict(X_query)[0]

pd.DataFrame(
    {
        "prediction (no scaling)": [pred_plain],
        "prediction (with scaling)": [pred_scaled],
        "nearest neighbour (no scaling)": [
            train_idx[
                knn_plain.kneighbors(
                    X_query, return_distance=False
                )[0, 0]
            ]
        ],
        "nearest neighbour (with scaling)": [
            train_idx[
                knn_scaled["knn"].kneighbors(
                    knn_scaled["scaler"].transform(X_query),
                    return_distance=False,
                )[0, 0]
            ]
        ],
    },
    index=["a"]
)
```

	prediction (no scaling)	prediction (with scaling)	nearest neighbour (no scaling)	nearest neighbour (wit
a	520000.0	390000.0	b	c

Takeaway:

- **Unscaled:** a b prediction **$520,000**

- **Scaled:** a c prediction **$390,000**

The query has no known target, so this is not a performance comparison. It demonstrates that the chosen representation changed the nearest neighbour—and therefore the prediction.

Why this happens

- Squared Euclidean distance aggregates per-feature squared differences:

$$\|u - v\|_2^2 = \sum_j (u_j - v_j)^2.$$

- A large-scale feature (e.g., **sqft**) can dwarf small-scale features (e.g., **rooms**), so KNN effectively "ignores" the smaller-scale dimensions.
- **Standardization** (z-scores) or **min-max scaling** puts dimensions on comparable footing.
- **Rule of thumb:** For distance-based methods, scale features unless their existing units deliberately encode the relative importance that the distance should use.

Distances to the training examples

Distances from **a** to {**b**, **c**} before and after scaling.

```
def show_pair(name_from, names_to, D):
    return D.loc[name_from, names_to].to_frame("distance")

print("Unscaled distances from a → {b,c}")
display(show_pair("a", ["b", "c"], dist_unscaled))

print("Scaled distances from a → {b,c}")
display(show_pair("a", ["b", "c"], dist_scaled))
```

```
Unscaled distances from a → {b,c}
```

	distance
b	20.396078
c	200.000000

```
Scaled distances from a → {b,c}
```

	distance
b	2.008247
c	1.818182

Switch to Manhattan distance?

Scale also matters with L_1 distance:

$$|u - v|_1 = \sum_j |u_j - v_j|.$$

The same training data can be reused with `metric="manhattan"`.

```
# 1) No scaling
knn_manhattan_plain = KNeighborsRegressor(
    n_neighbors=1, metric="manhattan"
)
knn_manhattan_plain.fit(X_train, y_train)
pred_manhattan_plain = knn_manhattan_plain.predict(X_query)[0]

# 2) With scaling (pipeline fits scaler only on training, then KNN on scaled)
knn_manhattan_scaled = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsRegressor(n_neighbors=1, metric="manhattan"))
])
knn_manhattan_scaled.fit(X_train, y_train)
pred_manhattan_scaled = knn_manhattan_scaled.predict(X_query)[0]

pd.DataFrame(
    {
        "prediction (no scaling)": [pred_manhattan_plain],
        "prediction (with scaling)": [pred_manhattan_scaled],
    },
    index=["a"]
)
```

	prediction (no scaling)	prediction (with scaling)
a	520000.0	390000.0

TL;DR

- **Distance-based models** are sensitive to the units and relative scales of their features.
- Fit a scaler on the **training data only**, then reuse it for validation, test, and production examples.
- A pipeline keeps scaling and prediction together and prevents leakage during cross-validation.
- Scaling changes the model's geometry and can therefore change nearest neighbours and predictions.